

03 — Natural Language → SQL (HANA AI Query)

What it is: lets a user ask a plain-English question and get back a pandas DataFrame, by having the LLM write HANA SQL against a schema you supply — with a safety net so it can never run anything but SELECT.

Flow

user question + table schema → LLM generates SQL → safety check → execute → DataFrame

Code Pattern

```
from langchain.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

SQL_PROMPT = ChatPromptTemplate.from_template("""
You are an expert SAP HANA SQL developer.
Rules:
- Write ONLY a single SELECT statement. No INSERT/UPDATE/DELETE/DROP/DDL.
- Use only tables/columns in the schema below.
- Use HANA syntax: FETCH FIRST N ROWS ONLY for "top N".
Schema: {schema_context}
Question: {user_query}
SQL Query:
""")

def generate_sql(user_query, schema_context, llm) -> str:
    chain = SQL_PROMPT | llm | StrOutputParser()      # LangChain pipe syntax: prompt -> model -> parser
    raw = chain.invoke({"schema_context": schema_context, "user_query": user_query})
    return raw.strip().strip("`")                  # strip stray markdown fences

def execute_sql(conn, sql: str) -> "pd.DataFrame":
    first_word = sql.strip().split()[0].upper()
    if first_word != "SELECT":                      # <- the safety guard
        raise ValueError(f"Only SELECT queries are allowed. Got: '{first_word}'")
    cursor = conn.cursor()
    cursor.execute(sql)
    rows, cols = cursor.fetchall(), [d[0] for d in cursor.description]
    cursor.close()
    import pandas as pd
    return pd.DataFrame(rows, columns=cols)
```

Full Agent (wires schema + generate + execute together)

```
class QueryAgent:
    def __init__(self, conn, tables, llm=None):
        self.conn = conn
        self.llm = llm or get_llm()
        self.schema_context = build_schema_context(conn, tables)    # SYS.COLUMNS lookup

    def ask(self, user_query: str):
        sql = generate_sql(user_query, self.schema_context, self.llm)
        return execute_sql(self.conn, sql)

agent = QueryAgent(conn, tables=["SALES_ORDERS"])
df = agent.ask("Top 5 customers by total order value")
```

Key Points

- **Two layers of safety:** the prompt instructs SELECT-only, AND `execute_sql` independently rejects any non-SELECT first word — never trust the LLM's instruction-following alone.
- `schema_context` comes from querying HANA's `SYS.COLUMNS` system view — this is what lets the LLM know real column names instead of guessing.

- temperature=0 on the LLM for SQL generation — you want the same question to reliably produce the same SQL.
- FETCH FIRST N ROWS ONLY is HANA's syntax for limiting rows (not LIMIT like MySQL/Postgres, not TOP N in the SELECT list like SQL Server — though HANA also accepts SELECT TOP N).

Interview Q&A

Q: Why not just trust the LLM to only generate SELECT statements? A: LLMs can be prompt-injected or simply make mistakes. Defense in depth — validate the actual SQL string before it ever touches the database, regardless of what the model was told to do.

Q: What happens if the LLM references a column that doesn't exist? A: execute_sql lets HANA's own error surface (SQL execution failed: ...) — the agent doesn't try to silently fix or guess; the caller sees a clear DB error to react to.

Q: How would you extend this to support multiple tables with joins? A: Pass multiple table names into build_schema_context (or explicit {"table":..., "schema":...} dicts) so the LLM sees every column across all tables and can write the JOIN itself.